

Les pièges de ce projet

Tout ce qui a mordu, ou qui a failli. Chaque entrée dit le piège, ce qu'il produit, et la parade retenue.

À quoi ça sert. La plupart de ces pièges ne se voient pas à la relecture du code : ils se manifestent en conditions réelles, souvent en silence, et quelques-uns auraient rendu le boîtier inutilisable un jour de tournage. Les consigner évite de les redécouvrir.

Les plus coûteux sont marqués . Ce sont ceux qui échouent SANS MESSAGE : ce sont toujours les pires, parce qu'on cherche au mauvais endroit.

Sommaire

Sommaire	1
1. Le boîtier et son système	1
2. Le logiciel	4
3. L'interface	5
4. Les données et les formats	7
5. L'outillage et la vérification	10
6. L'usage, le jour du tournage	11
Ce qu'on en retient	12

1. Le boîtier et son système

Un chemin entre guillemets dans une unité systemd

`WorkingDirectory="$PROJECT_DIR"` : systemd **ne retire pas les guillemets** dans ce réglage. Il reçoit un chemin qu'il juge non absolu et refuse toute l'unité, avec un message que personne ne peut interpréter : *bad unit file setting*. Passait sur Bookworm, refusé sur trixie. **Parade** : pas de guillemets. Les `Exec*`, eux, les acceptent.

`set -e` qui coupe sur un message incompréhensible

Quand le service refusait de démarrer, le script d'installation s'arrêtait net — et le **WiFi, le pare-feu et le kiosque n'étaient jamais configurés**. L'utilisateur ne voyait qu'une ligne obscure. **Parade** : afficher `systemctl status` et `journalctl` avant de sortir.

XAUTHORITY manquant : une fenêtre qui ne s'ouvre jamais, sans erreur

Un programme graphique lancé depuis un service systemd est **refusé par le serveur graphique** si cet en-tête d'autorisation manque. `DISPLAY` seul ne suffit pas. Rien ne s'affiche, rien ne le dit. **Parade** : résoudre `XAUTHORITY` en même temps que `DISPLAY`.

Le chemin du fichier PID dérivé de l'environnement

La session graphique et le service systemd n'ont pas le même `HOME` ni le même `XDG_RUNTIME_DIR`. Un chemin construit à partir de l'un donne **deux fichiers différents**, donc un état faux en permanence.

Parade : un chemin fixe, `/tmp/prompteur-kiosk-<uid>.pid`.

`PrivateTmp=yes` casse ce partage

Excellent réglage de sécurité en général — **ici il isole le service dans son propre** `/tmp`, et il perd de vue le navigateur du kiosque. **Parade** : tout le durcissement systemd, sauf celui-là, et le dire en commentaire.

Scripts enregistrés en mode 644 alors qu'on fait `chmod +x`

La procédure fait `chmod +x install/setup.sh`. Git voit ce changement de permission comme une modification locale, et **bloque tout** `git pull` ultérieur. **Parade** : enregistrer les scripts en 755 dans le dépôt.

Fins de ligne CRLF sur un fichier `.sh`

Un script écrit depuis Windows avec des fins de ligne CRLF échoue sur le Pi avec *bad interpreter*. Les outils d'édition le font sans prévenir. **Parade** : `.gitattributes` force LF, et on vérifie après chaque écriture automatisée.

`su` dans un script qui tourne déjà sous le bon utilisateur

Ajouté pour lire le dossier du bureau : `su` **réclame un mot de passe** et bloque l'installation sur une invite que personne n'attend. **Parade** : ne pas changer d'utilisateur quand on est déjà le bon.

Le dossier du bureau ne s'appelle pas « Desktop »

Sur un système en français, c'est « Bureau ». **Parade** : `xdg-user-dir DESKTOP`, avec repli sur les deux noms.

SSH est désactivé d'usine sur Raspberry Pi OS

La première tentative de connexion au boîtier a échoué, et on a d'abord soupçonné le câble, puis le pare-feu du projet. Ni l'un ni l'autre : **le service SSH n'était tout simplement pas démarré**, comme sur toute installation Raspberry Pi OS. Le pare-feu, lui, ne ferme que le port du prompteur — il laisse le port 22 tranquille, et l'a toujours fait. **Parade** : `install/setup.sh` l'active explicitement. Et quand un accès réseau échoue, vérifier d'abord que le service écoute, avant de soupçonner ce qui est entre les deux.

Changer le nom de la machine sans changer `/etc/hosts`

`hostnamectl set-hostname` renomme la machine, mais laisse `/etc/hosts` pointer sur l'ancien nom. Le système ne sait alors plus résoudre son propre nom, et **chaque** `sudo` **attend dix secondes** avant de rendre la main. Rien n'échoue, rien ne s'affiche : la machine devient seulement inexplicablement lente, et on cherche du côté du matériel. **Parade** : les deux vont ensemble, toujours. Le script remplace l'ancien nom dans `/etc/hosts` dans la foulée.

Deux machines derrière une box ne peuvent pas se joindre, quel que soit le logiciel

La demande était légitime : « pas de serveur externe, tout doit tourner dans le Raspberry ». Elle se heurte à une contrainte de réseau, pas de logiciel. Une box refuse les connexions qui arrivent d'internet ; si le boîtier ET l'ordinateur de la personne qui aide sont chacun derrière la leur, aucun des deux ne peut appeler l'autre. **Parade** : le boîtier appelle un **relais** — une connexion sortante, que la box laisse passer comme une page web — et le relais met les deux en relation. Tous les outils de prise en main à distance fonctionnent ainsi, TeamViewer compris ; la seule question est **à qui appartient le relais**. Raspberry Pi

Connect fournit le sien, officiel et gratuit. L'expliquer par l'image des deux téléphones qui ne savent qu'appeler a débloqué la discussion mieux qu'un schéma de réseau.

Une commande « évidente » absente de la documentation officielle

`rpi-connect on` y figure ; `rpi-connect off` et `rpi-connect signout` n'y figurent pas, et la sortie de `rpi-connect status` n'y est décrite nulle part. Le script s'appuyait sur les trois, de mémoire. Rien ne permettait de les essayer : le boîtier est à distance, et l'outil ne tourne pas sur le poste de développement. **Parade** : vérifier chaque commande contre la documentation AVANT de l'écrire, et construire autour de ce qui est confirmé. Pour couper l'accès, le script lance aussi `vnc off` et `shell off`, documentées — écran et terminal sont fermés même si `off` n'existait pas. Et l'analyse de `status`, non garantie, ne sert plus qu'à éviter un lien superflu : **aucun message n'annonce un échec sur sa seule foi**, on montre l'état réel.

Un interrupteur d'accès à distance ne va pas dans la page web

Le plus pratique aurait été un bouton « activer l'assistance » dans la télécommande. Mais cette page est ouverte à **tout téléphone connecté au WiFi du boîtier** : n'importe qui à portée, sur un plateau, pourrait ouvrir la vue de l'écran — texte du prompteur compris — à une personne extérieure. **Parade** : l'interrupteur est une **icône du bureau du boîtier**, donc exige d'être devant lui. Coupé par défaut, ouvert par le journaliste, refermé par lui. La commodité se paie en sécurité ; ici, elle ne valait pas ce prix.

Relancer le grand script pour obtenir une icône

Le boîtier du journaliste venait d'être installé. Pour lui ajouter les icônes d'assistance, la voie naturelle était de relancer `setup.sh` — qui **recrée le WiFi**, avec le piège du mot de passe régénéré. **Parade** : chaque fonctionnalité doit pouvoir s'ajouter **seule**. Les icônes sont posées par `assistance.sh` installer, que `setup.sh` appelle aussi : une seule définition, et un boîtier déjà en service les obtient sans rien risquer.

« À distance » ne veut pas dire « depuis n'importe où »

J'ai activé SSH et écrit, en tête de deux guides, « c'est la dernière fois que vous vous déplacez » et « les mises à jour suivantes se font de chez vous ». Le corps des mêmes guides disait pourtant l'exact contraire : il faut être relié à **la même box** que le boîtier. Une box refuse les connexions qui viennent d'internet — c'est son rôle. Depuis un autre lieu, rien ne passe. La contradiction était dans les **phrases d'accroche**, c'est-à-dire précisément celles qu'on lit, et elle a été découverte au moment où la question devenait concrète : « la personne a réussi l'installation, je peux m'y connecter ? » **Parade** : une promesse en tête de document se vérifie contre la **condition la plus restrictive** écrite plus bas, pas contre le cas favorable qu'on avait en tête. Et « accessible » doit toujours préciser **depuis où**.

Une adresse distribuée par une box n'est pas une adresse

Documenter « connecte-toi à 192.168.1.42 » revient à documenter une valeur qui change au prochain rebranchement. L'utilisateur suit la procédure à la lettre, et elle ne marche plus. **Parade** : un nom, pas un nombre. `avahi-daemon` + un nom de machine fixe donnent `prompteur.local`, qui se résout sans rien installer depuis Windows, macOS et Linux. Et pour le cas où même cela échoue, le WiFi du boîtier offre une adresse qui, elle, ne dépend de personne : `10.42.0.1`.

raspi-config change de présentation

Les versions récentes scindent « Boot / Auto Login » en **deux lignes distinctes**. N'en faire qu'une laisse le boîtier sur un écran de connexion, et le prompteur ne démarre jamais seul. **Parade** : documenter les deux présentations, et dire que les deux réglages sont nécessaires.

Le pilote d'écran fourni par le vendeur

Le script `LCD35C-show` **désactive le pilote graphique** `vc4-kms-v3d` — sur un Pi 5 il n'y a aucun mode de secours derrière — et **force le HDMI en 480×320**, écrasant le grand écran. Il est conçu pour une machine à un seul écran. **Parade** : ne pas le lancer. Déclarer l'écran en mode DRM, à la main.

Les réglages `hdmi_*` sont ignorés sur Pi 5

Presque tous les tutoriels en ligne datent d'avant le changement d'architecture graphique. Les recettes qu'on trouve **ne font rien**, sans le dire.

2. Le logiciel

Un texte trop gros fige l'écran, et le gel survit au redémarrage

Aucune borne sur la taille. Un `.docx` de 35 Ko dont le XML se décompresse en 8,9 Mo donnait **7,7 Mo de texte et 40 000 lignes** ; l'affichage crée un élément par ligne. Et comme le texte est enregistré, **l'écran restait figé à chaque démarrage** : seule sortie, un clavier et un terminal. Ce n'est pas qu'une attaque : un PDF de catalogue importé par erreur fait pareil. **Parade** : borner la taille décompressée avant de lire, puis le texte, puis tronquer au chargement — le boîtier doit redémarrer, même dégradé.

Le confinement réseau ne protège pas du navigateur

Le pare-feu limite le port au WiFi du boîtier. Mais une page web piégée ouverte sur un téléphone **déjà connecté à ce WiFi** peut viser le boîtier sans la moindre interaction : la requête part de l'intérieur du périmètre. **Parade** : contrôle de l'origine, `application/json` exigé, en-tête maison sur l'envoi de fichier.

Un GET qui modifie l'état

`/api/library/load` répondait en GET tout en changeant le texte à l'antenne : une simple balise `<img>` suffisait. **Parade** : toute route qui modifie quelque chose est en POST.

Jinja ne recharge pas les gabarits hors mode debug

Un `git pull` qui change une page **n'a aucun effet** tant que le service n'est pas redémarré. On croit la mise à jour ratée. **Parade** : `sudo systemctl restart prompteur` fait partie de la procédure, en gras.

La politique de sécurité interdit les scripts écrits dans la page

Ajouter une CSP `script-src 'self'` casse instantanément tout `<script>` inline — ici celui qui indiquait le mode de l'écran. **Parade** : passer la donnée par un attribut `data-`. Un attribut est une donnée, pas du code.

Un réglage refusé qui passe pour appliqué

Le serveur répondait « ok » même quand rien n'avait été accepté. Le bouton se colorait, on croyait avoir changé de mode, **rien n'avait bougé**. **Parade** : répondre 400 en nommant les clés refusées, et relire l'état réel.

Chaque appui sur Lecture renvoyait tout le script

La commande faisait avancer la version générale, ce qui obligeait chaque écran à retélécharger l'intégralité du texte — plusieurs mégaoctets sur le WiFi, à chaque bouton, au pire moment. **Parade** : ne faire avancer la version que quand un réglage change réellement.

Un `state.json` écrit sans synchronisation

Le renommage est atomique, mais les données peuvent n'être pas encore sur la carte lors d'une coupure. Le fichier tronqué est alors **écarté en silence** : le journaliste rallume et retrouve le texte de bienvenue, sans explication. **Parade** : `flush + fsync` avant de renommer.

Un verrou sans expiration condamne l'appareil

Réserver la vue Journaliste (l'écran principal) à un seul appareil est nécessaire. Mais un verrou qu'on oublie de rendre — onglet fermé brutalement, WiFi coupé — **interdirait à tout le monde de lire**, soit l'inverse du but. **Parade** : un bail qui expire seul, et une reprise en main toujours offerte.

Une suppression sans recours

La croix effaçait un texte sans confirmation, sans corbeille, sans sauvegarde, et le dossier des textes n'est pas versionné. **Parade** : confirmation, et l'échec est dit au lieu d'être avalé.

Une commande qui ne connaît qu'un mode

Lecture et Pause, sur le téléphone, agissaient sur un drapeau que **seul le mode Maintien lisait**. En Impulsion et en Dynamique, les boutons ne faisaient rien — sans le moindre message. C'est le journaliste qui l'a signalé. **Parade** : un seul point d'entrée par geste (lecture, arrêt, début), qui connaît les trois modes ; et un banc d'essai qui joue chaque geste dans chaque mode.

La dernière commande rejouée à l'ouverture

Un écran qui s'ouvre découvre la dernière commande envoyée et l'applique : un « Lecture » de la veille faisait défiler le texte tout seul au démarrage du boîtier. **Parade** : à l'ouverture, on note le numéro de la dernière commande sans l'exécuter.

Une lettre abîmée empêchait le boîtier de démarrer

Le chargement de `state.json` rattrapait un JSON invalide, mais pas un octet non UTF-8 — ce qu'une coupure de courant peut laisser. Le service plantait au démarrage, redémarrait, replantait : écran noir à chaque allumage. **Parade** : rattraper `ValueError`, qui couvre les deux.

Un navigateur lancé par le service meurt avec lui

Ouvert depuis Settings (changement de vue, Échap), le navigateur du grand écran est un enfant du service : tout redémarrage du service le fermait, et le grand écran retombait sur le bureau. **Parade** : au démarrage, le serveur rouvre la vue qui était affichée (`kiosk.sh --reprendre`) — sauf si elle avait été fermée volontairement.

Un kiosque relancé trouve sa place prise par son propre fantôme

Fermer puis rouvrir le navigateur du kiosque (changement de vue du grand écran) laissait la place de meneur à l'ancienne page pendant 12 s. La nouvelle affichait « déjà en cours » — et son voile ne partait jamais, même la place libérée. **Parade** : le serveur rend la place du kiosque quand il le ferme lui-même ; et un écran écarté reprend la main dès que le serveur la lui rend.

3. L'interface

Le plein écran est refusé sans geste de l'utilisateur

Un navigateur **refuse toujours** le plein écran demandé au chargement. La fonction semble cassée alors qu'elle est simplement interdite à ce moment-là. **Parade** : l'installer au premier geste venu ; et ouvrir l'écran principal dans une fenêtre dédiée, ce qui donne le geste.

En plein écran, Échap appartient au navigateur

Il s'en sert pour sortir du plein écran : le gestionnaire de la page **ne la voit jamais**. **Parade** : écouter la sortie de plein écran et la traiter comme la demande de quitter.

Affecter l'état APRÈS avoir demandé le réaffichage

La fonction de chargement écrivait `setText(...)` puis `marks = ...`. Or c'est `setText` qui redessine la zone de saisie, et il lit `marks`. L'éditeur affichait donc la mise en forme du texte PRÉCÉDENT — et **aucune au tout premier chargement**, quand la liste est encore vide. Rien ne signalait l'erreur : le texte, lui, était juste. **Parade** : affecter tout ce dont dépend un rendu AVANT de le déclencher. Le défaut a survécu plusieurs semaines parce qu'on vérifiait la mise en forme sur l'écran de lecture, où elle était correcte, et jamais sur la zone de saisie au premier chargement.

Un bouton prend le focus au clic, et la sélection disparaît

Appliquer un style au texte sélectionné échoue dans certains navigateurs, parce que le bouton a vidé la sélection avant même le clic. **Parade** : `mousedown + preventDefault` : le bouton ne prend pas le focus.

`window.close()` n'est autorisé que sur une fenêtre ouverte par script

Une page ordinaire ne peut pas se fermer elle-même. **Parade** : ouvrir l'écran de lecture par `window.open`, ce qui autorise ensuite à le refermer — et évite la navigation, donc l'impasse.

Quitter vers une page morte, sans flèche retour

En kiosque il n'y a **ni barre d'adresse ni bouton retour**. Si le service est tombé, revenir à l'accueil mène à une page d'erreur sans aucun moyen d'en sortir. **Parade** : refuser de quitter tant que la liaison est perdue, et l'expliquer.

Reconstruire la zone de saisie à chaque frappe

Cela replace le curseur au début à chaque caractère. **Parade** : recalculer les données sans reconstruire ; le rendu suit au prochain geste explicite.

Une fonction complète et pourtant inutilisable

La mise en forme s'appliquait correctement, mais **rien ne le montrait** — une zone de saisie ordinaire ne sait pas afficher du gras. On concluait que le bouton ne faisait rien. **Parade** : montrer le résultat là où l'on agit. C'est ce qui a conduit à remplacer la zone de saisie par une zone éditée riche.

Un bouton désactivé qui n'en a pas l'air

« Ouvrir » restait vert vif alors qu'il ne faisait rien : on le presse en boucle. **Parade** : styliser l'état désactivé.

Deux écrans principaux se disputent le défilement

Chacun pousse sa position : le texte saute d'un endroit à l'autre en pleine lecture. **Parade** : une seule vue Journaliste à la fois ; une seconde affiche un voile et propose la vue Spectateur ; la reprise en main reste possible.

Suivre au pixel près désigne un autre passage sur un autre écran

La vue Spectateur recopiait la position du meneur en pixels. Sur un écran d'une autre taille, les lignes ne se coupent pas au même endroit : la régie lisait une autre phrase que le journaliste. **Parade** : le meneur envoie la LIGNE du texte sous la ligne rouge et la fraction déjà passée ; chaque écran la replace dans sa propre mise en page. Le même repère garde la phrase quand on change la taille du texte en pleine lecture.

Un raccourci d'écran d'accueil ouvre la page de son manifeste

« Ajouter à l'écran d'accueil » n'enregistre pas la page affichée, mais l'adresse de départ déclarée par le manifeste. Un manifeste unique, pointant sur l'écran de lecture, faisait ouvrir au raccourci de la télécommande... l'écran de lecture, déjà tenu par le boîtier. **Parade** : un manifeste par vue.

4. Les données et les formats

Des indices calculés avant nettoyage sont faux après

La mise en forme d'un document Word se lit sur le XML brut, mais elle doit désigner le texte **après** nettoyage — or le nettoyage change les longueurs. Une plage décalée de trois signes **met en évidence les mauvais mots** en pleine lecture : c'est pire que pas de mise en forme du tout. **Parade** : un nettoyage qui note, pour chaque caractère survivant, d'où il vient. La table est produite pendant le nettoyage, jamais reconstituée après coup.

Un préfixe retiré à l'affichage compte encore dans les indices

L'écran retirait [centre] d'une ligne avant de l'afficher, puis avançait son compteur de la longueur AFFICHÉE. Toutes les plages des lignes suivantes glissaient de quelques lettres : le gras et les couleurs tombaient sur les mauvais mots. **Parade** : avancer de la longueur de la ligne BRUTE ; le banc d'essai le vérifie.

Un nettoyage est contextuel, pas local

Découper d'abord puis nettoyer chaque morceau donne un texte **différent** : deux espaces à cheval sur une frontière ne se voient plus. **Parade** : un seul algorithme de nettoyage, pour tous les cas.

Stocker du HTML pour de la mise en forme

Tenant, et c'est le piège : il faut alors se fier à un échappement partout, et un ancien fichier devient ambigu. **Parade** : le texte reste une chaîne brute, la mise en forme est une liste de plages posées dessus. Rien à échapper, et les anciens textes s'affichent inchangés.

Un plafond qui s'applique en silence

La mise en forme est bornée à 500 plages, parce que le calcul du style est quadratique en nombre de plages et qu'un Pi n'y survivrait pas au-delà. Mais sur un document long, l'écrtage était **muet** : la fin du texte arrivait sans mise en forme, et rien ne distinguait ce cas d'un import raté. **Parade** : la borne reste, le silence non. L'import renvoie `marksTruncated`, et la télécommande le dit en clair. Une limite annoncée est une limite ; une limite tue est un bogue que l'utilisateur découvre seul, en tournage.

Un format qui ne déclare rien oblige à tout déduire

Un PDF ne dit jamais « ce mot est en gras ». Il dit « ce mot est peint avec la police Arial-BoldMT », à telle position, avec telle couleur de remplissage. Chercher une propriété « gras » n'aurait rien donné et aurait fait conclure, à tort, que le format ne porte aucune mise en forme. **Parade** : déduire de ce que le format dit réellement — le **nom de la police** pour le gras et l'italique, l'**opérateur de remplissage** pour la couleur, la **position de la ligne** pour le centrage, et « ligne entièrement plus grosse que le corps » pour un titre.

« Impossible » voulait dire « pas avec la méthode que j'employais »

J'ai écrit noir sur blanc, dans ce document et dans le mode d'emploi, que le souligné d'un PDF n'était **pas récupérable** : ce n'est pas un attribut du texte, c'est un trait tracé dessous, un objet graphique sans lien déclaré avec les mots. Tout cela est exact. La conclusion, elle, était fausse — je cherchais une **propriété**, alors que la réponse était **géométrique** : relever les traits horizontaux, et rapprocher chacun de la ligne de texte qui passe juste au-dessus. **Parade** : ne pas confondre « le format ne le déclare pas » et « on ne peut pas le savoir ». Avant d'inscrire une limite dans la documentation, chercher ce que le format dit **réellement**, et non ce qu'on espérait y trouver. Une limite écrite trop vite se recopie ensuite dans tous les documents, et personne ne la rouvre.

La position du texte n'est pas dans la matrice du texte

Word et LibreOffice ne déplacent pas le curseur de texte : ils déplacent le **repère**. Deux lignes distinctes portent alors exactement la même position de texte, et seule la matrice courante les sépare. Le code ne lisait que la position du texte : toutes les lignes d'un vrai document se retrouvaient à la même hauteur, donc fondues en **une seule ligne**. Rien ne le signalait — le texte restait juste, seuls l'alignement et le soulignement tombaient à côté. **Parade** : composer la position avec la matrice courante. Et surtout : ne pas valider un lecteur de PDF sur un PDF fabriqué pour l'occasion, qui n'utilise pas les mêmes mécanismes qu'un document produit par un traitement de texte.

Une moyenne unique se trompe d'un mot entier

Pour situer un trait de soulignement dans une ligne, il faut convertir une distance en nombre de signes. Une largeur moyenne unique (une demi-chasse par signe) se trompait de deux à cinq caractères — c'est-à-dire d'un **mot entier** une fois recalé sur les limites de mots, et le soulignement se posait à côté. **Parade** : deux gestes qui se complètent. Distinguer les signes étroits (i, l, espace, ponctuation) des larges (m, w, majuscules) ramène l'erreur à un signe au plus ; le recalage sur les limites de mots absorbe ce qui reste. Aucun mot n'est souligné à moitié.

Reconstruire une ligne autrement que la bibliothèque ne l'a rendue

Les indices du soulignement étaient calculés sur le **recollage des morceaux** observés, alors que le texte affiché venait de la bibliothèque, qui insère des espaces entre eux. Trois espaces d'écart suffisaient à décaler le résultat d'un mot. **Parade** : un seul repère pour tout le monde — situer d'abord chaque morceau dans le texte produit, puis raisonner uniquement dans ces coordonnées-là.

Ajouter sans reconstruire

La tentation, en lisant les morceaux d'un PDF un par un, est de recoller le texteso-même à partir d'eux. On perd alors tout le travail d'espacement et de découpage en lignes de la bibliothèque — et le texte, lui, **régresse** pendant qu'on croit l'enrichir. **Parade** : laisser l'extraction existante produire le texte, et **reposer** la mise en forme dessus en cherchant chaque morceau dans l'ordre. Un test verrouille l'invariant : aux dièses de titre près, le texte produit est identique à celui d'avant.

Un nom qui diffère entre le format et le modèle

Le RTF écrit le souligné `\u1`. Le modèle interne, lui, l'appelle `u`. Le code écrivait `etat[mot]`, donc `etat["u1"]` — une clé que personne ne lisait. Le souligné disparaissait sans la moindre erreur : tout le reste fonctionnait, et l'oubli portait sur un seul des trois styles. **Parade** : traduire explicitement au point d'entrée, et tester **chacun** des styles séparément. Un test qui ne vérifie que le gras aurait laissé passer celui-ci.

Les tables internes d'un format ne doivent jamais atteindre l'écran

Un RTF commence par sa table des polices, sa table des couleurs, sa feuille de styles et les propriétés du document — titre, auteur, logiciel générateur. Tout cela est du texte, entre accolades, au même titre que le script. Sans mise à l'écart explicite, le prompteur aurait affiché « Times New Roman;Symbol;red255green0blue0 » avant la première phrase. **Parade** : une liste des groupes à ignorer, appliquée à la profondeur d'accolade, et un test qui vérifie l'ABSENCE de ces contenus — pas seulement la présence du bon texte.

Une limite de mot qui accepte le tiret

`<text:list` suivi d'une limite de mot reconnaît aussi `<text:list-item`, puisque le tiret n'est pas un caractère de mot. Mais `</text:list>` ne reconnaît pas `</text:list-item>` : chaque élément ouvrirait donc une liste que rien ne refermait, et **tout le document à partir de la première liste se retrouvait à puces**. **Parade** : `(?=[\s/>])` au lieu de la limite de mot, et un test de non-régression qui vérifie ce qui suit la liste, pas seulement la liste.

Une référence calculée sur les seules valeurs écrites

Pour savoir si un passage est « plus gros que le reste », il faut la taille du corps du texte. La prendre comme la plus répandue parmi les tailles écrites donnait le contraire du résultat voulu : dans un document où un seul mot est agrandi, ce mot est la seule taille écrite — donc la référence — donc il n'a plus rien de spécial. **Parade** : compter aussi les passages SANS taille écrite, pour la valeur par défaut. Ce qui n'est pas écrit fait partie du décompte.

Recopier fidèlement peut rendre invisible

Reproduire à l'identique la couleur d'un document semble être la bonne réponse à « que ça rende comme l'original ». Sur un prompteur à fond noir, un bleu marine fidèlement recopié devient illisible — au moment précis où l'on comptait sur le passage mis en avant. **Parade** : garder l'INTENTION et non la valeur. La correspondance se fait sur la teinte, vers une palette fermée dont toutes les couleurs sont lisibles. Même raisonnement pour la taille, gardée relative : c'est le lecteur qui fixe la taille générale selon sa distance.

Une liste blanche posée sur un seul des deux chemins d'entrée

L'import par clé USB vérifiait l'extension du fichier ; l'envoi depuis le téléphone, non. Une photo choisie par erreur ne donnait aucun message : elle tombait dans le décodage texte de secours et remplissait la zone de saisie d'octets illisibles, avec un **code 200** pour dire que tout allait bien. **Parade** : la vérification descend dans `extract_rich()`, là où passent **tous** les chemins d'entrée, plutôt que d'être recopiée dans chaque appelant. Un test vérifie aussi l'inverse : qu'aucun format promis au client n'est refusé.

Un signe qui marche à l'import mais pas au clavier passe pour une panne

Le tiret de puce était converti à l'import et nulle part ailleurs. Tapé dans la zone de saisie, il ne produisait rien — alors que l'aide, écrite par mes soins, affirmait « si vous préférez taper, ces signes font la même chose ». Le journaliste a signalé une panne ; le code faisait exactement ce que j'avais écrit, et c'est la promesse qui était fausse. Le dièse de titre, lui, marchait depuis toujours aux deux endroits — parce qu'il est traduit **au rendu**, et non converti à l'entrée. **Parade** : une règle, pas deux. Les marqueurs de DÉBUT DE LIGNE (`#`, `-`, `[centre]`) ne sont jamais convertis : ils restent dans le texte et l'écran les traduit. Ceux qui ENTOURENT un passage (`**gras**`, `[rouge]...[/rouge]`) sont convertis à l'import, et la zone de saisie a des boutons pour eux. La frontière est explicable en une phrase, donc elle est documentée en une phrase — et ce qui reste dans le texte reste effaçable, ce qui compte d'autant plus qu'aucun bouton ne permet de centrer une ligne.

Vérifier une promesse de documentation comme on vérifie du code

L'écart n'a pas été trouvé par un test : aucun ne portait sur « ce que l'aide affirme ». Il a été trouvé par l'utilisateur, sur une capture d'écran. **Parade** : quand une aide dit « ceci marche ici », en faire un cas d'essai. Une phrase de documentation est une affirmation sur le comportement, au même titre qu'une assertion.

Un marqueur bavard vaut mieux qu'un marqueur malin

Pour la couleur et l'alignement en texte simple, aucune convention n'existe — il fallait en inventer une. La tentation est de faire court : `==texte==`, `->texte<-`. Deux défauts, et le second est le vrai : personne ne se souvient de ce que signifiait `==` six mois plus tard, et un signe court se déclenche par accident au milieu d'un texte ordinaire. **Parade** : écrire le marqueur **en toutes lettres** — `[rouge]...[/rouge]`, `[centre]`. C'est plus long à taper, et cette longueur est la fonctionnalité : elle rend le marqueur relisible sans mode d'emploi, et impossible à produire par hasard. Seuls les mots de la liste comptent, donc `[voir encadré]` et `[1]` traversent intacts.

J'avais écarté ce qui n'avait pas de convention, au lieu d'en créer une

Deux cases du tableau des formats sont restées vides une version de trop, avec une justification qui tenait debout : inventer une syntaxe obligerait à apprendre un langage pour faire moins bien que les boutons de la télécommande. C'est vrai pour un passage isolé, et faux pour quelqu'un qui prépare son texte ailleurs, dans un simple fichier — le cas même où le boîtier n'a aucun bouton à offrir. **Parade** : distinguer « ça n'a pas de sens » de « ça n'en a pas dans l'usage auquel je pensais ». Une case vide dans un tableau demande une raison qui tient pour TOUS les usages, pas seulement pour celui qu'on a en tête en l'écrivant.

Une syntaxe qui se déclenche toute seule peut abîmer un texte existant

Reconnaître ****gras**** dans un fichier texte rend service. Mais la même règle, appliquée sans précaution, transforme `3 * 4 = 12` en calcul mis en forme, `mon_fichier.txt` en mot souligné, et une adresse `http://x/a_b_c` en charabia — dans un script déjà écrit, sans que personne ne l'ait demandé. **Parade** : deux verrous, pas un. Seules les **paires soignées** comptent (le marqueur ouvre et ferme contre un signe visible, sur une même ligne) ; et la conversion n'a lieu qu'à l'**import d'un fichier**, jamais à la relecture d'un texte de la bibliothèque. Un texte enregistré ne peut donc pas changer d'aspect des mois plus tard. Les tests portent sur les pièges autant que sur les cas qui doivent marcher.

Une syntaxe de type markdown changerait les textes existants

Un `*` ou un `#` au milieu d'une phrase déjà enregistrée se mettrait soudain à signifier quelque chose.

5. L'outillage et la vérification

Filtrer l'affichage d'un contrôle masque son échec

`bandit` sort en erreur **dès la moindre alerte, même de sévérité faible**. En filtrant l'affichage sur « High » et « Medium », la ligne fautive n'apparaissait jamais : la CI est restée rouge **six commits d'affilée** pendant qu'on annonçait « tout est vert ». **Parade** : ce qui compte est le **code de sortie**, pas ce qu'on choisit d'afficher. `scripts/verifier-ci.sh` rejoue toutes les étapes sans filtre.

Un faux positif vaut mieux corrigé que muselé

`bandit` voyait un mot de passe en dur dans une clé de dictionnaire nommée `token`. **Parade** : renommer la clé. Un `nosec` aurait éteint le contrôle pour de bon.

Un outil qui analyse une liste de fichiers n'analyse pas le projet

Un nouveau module n'aurait jamais été examiné. **Parade** : analyser le dossier, avec des exclusions.

Ce qu'aucune règle n'interdit finira par arriver

Rien n'empêchait d'écrire du HTML depuis le JavaScript — le garde-fou qui manquait le plus une fois le contenu mis en forme introduit. **Parade** : l'interdire dans la CI. Un oubli ne se voit pas à la relecture ; il se voit à la CI.

Un réglage de test qui reste en place

Pour faire tenir une capture d'écran, la ligne de lecture a été coupée et le texte réduit par l'API. Ces réglages s'enregistrent dans `state.json`, que **tous les serveurs lancés depuis le dossier partagent** : l'utilisateur a rouvert le prompteur et cherché sa ligne rouge, disparue sans raison apparente. **Parade** : tout réglage modifié pour un essai est remis à sa valeur à la fin de l'essai, dans le même geste. `state.json` n'est pas versionné : le boîtier du journaliste, lui, n'était pas touché.

Un banc d'essai qui ment

Le volet de prévisualisation interne **bloque l'API plein écran et supprime les fenêtres popup**. Trois fonctions correctes ont semblé cassées, et il a fallu un essai dans un vrai navigateur pour trancher.

Parade : faire vérifier dans un vrai navigateur ce qui touche au plein écran, aux fenêtres et aux permissions.

Les outils d'édition automatisée abîment les échappements

Écrire du code par script en passant par un shell mange les `\n` et les antislashes, ce qui produit des fichiers corrompus, parfois subtilement. **Parade** : écrire les fichiers directement, et relire le résultat.

Un remplacement global finit par toucher ce qu'il ne fallait pas

Remplacer un appel partout a réécrit l'intérieur de la fonction qui le remplaçait : **réursion infinie**.

Parade : vérifier ce qu'on vient de remplacer, pas seulement que ça compile.

Une sauvegarde qui embarque des fichiers suivis par git

`tar ... scripts` emportait aussi les outils rangés dans `scripts/`. Restaurés par-dessus une version plus récente, ils bloquaient le `git pull` suivant. **Parade** : exclure `*.py` et `*.sh` à la sauvegarde comme à la restauration.

Une animation ne se teste pas dans une page masquée

Le navigateur ralentit ou suspend `requestAnimationFrame` quand la page n'est pas visible. Volet de prévisualisation masqué : les pédales « ne faisaient plus rien », alors que le code était juste. **Parade** : `tests/banc_pedales.js` fait tourner le vrai `display.js` avec une horloge qu'on avance à la main. Il est dans la CI.

6. L'usage, le jour du tournage

Le mot de passe WiFi n'est affiché qu'une fois

Généré au hasard à l'installation, il n'existe nulle part ailleurs. **Parade** : l'imposer soi-même à l'installation, et le noter sur la fiche du boîtier.

Relancer l'installation changeait le mot de passe WiFi

Sans redonner le mot de passe actuel, un nouveau était tiré au hasard et **plus aucun téléphone ne se connectait** — alors que deux guides faisaient relancer l'installation sans lui. Le réseau était en plus supprimé puis recréé ouvert, le temps de quelques commandes. **Parade** : `setup.sh` reprend le mot de passe EN PLACE, vérifie sa longueur avant de toucher au réseau, et modifie le réseau existant en une seule fois.

Pas d'internet une fois connecté au boîtier

C'est voulu. Mais le script doit donc **déjà être sur le téléphone** : on ne peut plus aller le chercher dans ses mails à ce moment-là. C'est ce qui bloque le plus sûrement un premier tournage.

Le logiciel du pédalier ne tourne que sous Windows

Parade : l'apprentissage des touches dans l'application, qui ne demande aucun PC.

Une touche de pédale peut rendre la pédale muette

F est prise par le plein écran, `Échap` par le retour à l'accueil, et deux pédales sur la même touche s'annulent. **Parade** : les refuser à l'apprentissage, et prévenir pour les autres raccourcis.

Un écran noir et muet

Quand la liaison est perdue, l'écran ne montrait rien et ne disait rien : personne ne pouvait savoir si le texte affiché était encore d'actualité. **Parade** : un bandeau qui dit ce qui compte — le texte reste lisible, les pédales fonctionnent.

Ce qu'on en retient

1. **Ce qui échoue en silence coûte le plus cher.** Sur les vingt pièges les plus coûteux de ce projet, la quasi-totalité ne produisait aucun message.
2. **Un garde-fou qui peut enfermer dehors est pire que pas de garde-fou.** Verrou sans expiration, sortie vers une page morte, bouton totalement inerte.
3. **Une fonction qu'on ne voit pas agir est réputée cassée**, même quand elle fonctionne.
4. **Vérifier, c'est regarder le code de sortie**, pas la sortie qu'on a choisi d'afficher.
5. **Le banc d'essai fait partie des sources d'erreur.** Quand un résultat surprend, suspecter l'outil de mesure autant que le code.